

Program ini dibuat untuk mengambil metadata dari halaman putusan di website Mahkamah Agung, lalu hasilnya disimpan ke dalam sebuah file CSV di Google Drive.

```
import requests
import urllib.request
import time
import os
import warnings
import re

from bs4 import BeautifulSoup

import csv
import os.path

from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

- requests → untuk request halaman web.
- re → regular expression, dipakai untuk membersihkan teks atau mencocokkan pola string.
- BeautifulSoup → parsing HTML.
- csv → menulis hasil ke file CSV.
- drive → mount Google Drive supaya file hasil bisa disimpan.

```
def generateFileCSV(listHasil, csvName1):

    csvFolder = "./"
    csvName = csvName1

    if os.path.exists(csvName):

        f = open(csvName, 'a', newline='\n') # jika file sudah ada → append
        print(f)
        print("ada")
        w = csv.writer(f)

    else:

        f = open(csvName, 'w', newline='\n') # jika file belum ada → buat baru
        print(f)
        print("tidak ada")

        w = csv.writer(f)

    # membuat atribut file csv
    w.writerow(("terdakwa", "penuntut_umum", "nomor", "tingkat_proses", "klasifikasi", "kata_kunci", "tahun", "tanggal",
                "lembaga_peradilan", "jenis_lembaga_peradilan", "hakim_ketua", "hakim_anggota", "panitera", "amar",
                "amar_lainnya", "catatan_amar", "tanggal_musyawarah", "tanggal_dibacakan", "kaidah", "abstrak", "url"))

    # menulis file csv
    for s in listHasil:
        w.writerow(s)

    f.close()
    berhasil = "\nCreate Csv file Berhasil\n"
    return berhasil
```

Fungsi generateFileCSV(listHasil, csvName1), fungsi ini digunakan untuk menulis hasil metadata ke file CSV.

```
if os.path.exists(csvName):
    f = open(csvName, 'a', newline='\n') # jika file sudah ada → append
    w = csv.writer(f)
else:
    f = open(csvName, 'w', newline='\n') # jika file belum ada → buat baru
    w = csv.writer(f)
    w.writerow(("terdakwa", "penuntut_umum", "nomor", ... , "url"))
```

Kalau file CSV sudah ada → data baru ditambahkan (append).

Kalau belum ada → buat file baru dan tulis header kolom.

Lalu setiap baris metadata (listHasil) ditulis ke CSV.

```
for s in listHasil:
    w.writerow(s)
```

```
def generateMeta(urlMeta):

    url = str(urlMeta).strip()
    response = requests.get(url, verify=False) # mengambil isi halaman web, tetapi tanpa memverifikasi SSL
    print(response)

    soup = BeautifulSoup(response.text, 'html.parser')
    cleanTags = re.compile('<.*?>') # regex untuk hapus tag HTML

    # metadata yang ingin diambil
    listMetaHead = ["terdakwa", "penuntut_umum", "nomor", "tingkat_proses", "klasifikasi", "kata_kunci", "tahun", "tanggal",
                    "lembaga_peradilan", "jenis_lembaga_peradilan", "hakim_ketua", "hakim_anggota", "panitera", "amar",
                    "amar_lainnya", "catatan_amar", "tanggal_musyawarah", "tanggal_dibacakan", "kaidah", "abstrak", "url"]

    # inialisasi
    listMeta = []

    rowsMETA1 = soup.find("ul", {"class": "portfolio-meta nobottommargin"}).find("table").findAll("tr")
    rowsMETA2 = soup.findAll("ul", {"class": "portfolio-meta nobottommargin"})

    # print('-----')
    for row in rowsMETA1:
        coll = row.findAll("td")

        cleantext2 = ''
        cleantext1 = ''

        if len(coll) > 1:
            cleantext2 = (re.sub(cleanTags, '', str(coll[1]))).strip()
            listMeta.append(cleantext2.replace('\n', ' '))
            # print(cleantext2.replace('\n', ' '))

        else:
            cleantext1 = (re.sub(cleanTags, ' ', str(coll[0]))).strip()
            # untuk putusan pidana akan muncul terdakwa dan penuntut umum pada meta
            # sedangkan untuk putusan selain pidana tidak muncul

            pidordpt = re.search( r'(.*)/Pdt.(.*)',str(cleantext1), re.M|re.I)

            # check dokumen putusannya pidana atau perdata
            if pidordpt == None: # artinya pidana
                entTerdakwah = re.search( r'(.*)Terdakwa:(.*)',str(cleantext1), re.M|re.I)
                entPenuntut = re.search( r'Penuntut Umum:(.*)Terdakwa:',str(cleantext1), re.M|re.I)
            else: # kalau perdata
                entTerdakwah = re.search( r'(.*)Tergugat:(.*)',str(cleantext1), re.M|re.I)
                entPenuntut = re.search( r'Pengugat:(.*)Tergugat:',str(cleantext1), re.M|re.I)

            if entTerdakwah == None:
                listMeta.append("")
            else:
                listMeta.append(entTerdakwah.group(2))

            if entPenuntut == None:
                listMeta.append("")
            else:
                listMeta.append(entPenuntut.group(1))

            # print(coll[0])
            # print(entTerdakwah.group(2))
            # print(entPenuntut.group(1))

    urlDL = rowsMETA2[1].findAll("li")
    urlDLStr = str(urlDL[4])
    listMeta.append(urlDLStr[urlDLStr.find("https"):urlDLStr.find('>')])
    #print(urlDLStr[urlDLStr.find("https"):urlDLStr.find('>')])
    #print(listMeta)

    return listMeta
```

Fungsi generateMeta(urlMeta), fungsi ini yang melakukan scraping metadata dari 1 halaman putusan.

Metadata yang ingin diambil sudah ditentukan:

```
listMetaHead = ["terdakwa", "penuntut_umum", "nomor", "tingkat_proses", "klasifikasi", "kata_kunci", "tahun", "tanggal_r",
                "lembaga_peradilan", "jenis_lembaga_peradilan", "hakim_ketua", "hakim_anggota", "panitera", "amar",
                "amar_lainnya", "catatan_amar", "tanggal_musyawarah", "tanggal_dibacakan", "kaidah", "abstrak", "url"]
```

Lalu ambil tabel metadata yang ada di dalam tag `<ul class="portfolio-meta nobottommargin">.`

```
rowsMETA1 = soup.find("ul", {"class": "portfolio-meta nobottommargin"}).find("table").findAll("tr")
rowsMETA2 = soup.findAll("ul", {"class": "portfolio-meta nobottommargin"})
```

Setiap baris (tr) diproses:

Kalau ada dua kolom (td), ambil isi kolom kedua (isi metadata).

Kalau hanya ada satu kolom, berarti itu berisi nama terdakwa dan penuntut. Karena bisa berbeda antara pidana dan perdata, maka dicek dulu dengan regex.

```
if pidorpdtd == None: # artinya pidana
    entTerdakwa = re.search(r'(.*)Terdakwa:(.*)', str(cleanText1))
    entPenuntut = re.search(r'Penuntut Umum:(.*)Terdakwa:', str(cleanText1))
else: # kalau perdata
    entTerdakwa = re.search(r'(.*)Tergugat:(.*)', str(cleanText1))
    entPenuntut = re.search(r'Penggugat:(.*)Tergugat:', str(cleanText1))
```

Kalau cocok, data terdakwa dan penuntut ditambahkan ke list. Kalau tidak ada, disimpan kosong.

Terakhir, ambil URL file putusan (PDF/Word) yang ada di bagian bawah halaman:

```
urlDL = rowsMETA2[1].findAll("li")
urlDLStr = str(urlDL[4])
listMeta.append(urlDLStr[urlDLStr.find("https"):urlDLStr.find('>')])
```

Hasilnya adalah list metadata satu putusan.

```
def main():

    warnings.filterwarnings('ignore')

    #folderListURL = pathFile
    fileListURL = "/content/drive/MyDrive/Semester_5/Information_Extraction/hasilListURLPage.txt"

    fileMetaCSV = "/content/drive/MyDrive/Semester_5/Information_Extraction/metaPidanaUmumPNwonogiri.csv"
    listHasil = []

    # waktu mulai
    startTime = time.time()

    # membuka file listURL
    openfileListURL = open(fileListURL, "r", encoding='UTF8')
    bacaListURL = openfileListURL.readlines()

    i = 1
    for barisURL in bacaListURL:
        try:
            #print(str(barisURL))
            hasil = generateMeta(str(barisURL))
            listHasil.append(hasil)
            print("===== ROW HASIL =====", i)
            #print(hasil)
        except Exception as e:
            print(f"Error Get Meta Inf, {e}")
            i=i+1

    createFile = generateFileCSV(listHasil, fileMetaCSV)

    openfileListURL.close()
    endTime = time.time()
    #print(listHasil)
    print('Time Processing : ', endTime-startTime, ' Second')

main();
```

```

===== ROW HASIL ===== 354
<Response [200]>
===== ROW HASIL ===== 355
<Response [200]>
===== ROW HASIL ===== 356
<Response [200]>
===== ROW HASIL ===== 357
<Response [200]>
===== ROW HASIL ===== 358
<Response [200]>
===== ROW HASIL ===== 359
<Response [200]>
===== ROW HASIL ===== 360
<Response [200]>
===== ROW HASIL ===== 361
<Response [200]>
===== ROW HASIL ===== 362
<Response [200]>
===== ROW HASIL ===== 363
<Response [200]>
===== ROW HASIL ===== 364
<Response [200]>
===== ROW HASIL ===== 365
<Response [200]>
===== ROW HASIL ===== 366
<Response [200]>
===== ROW HASIL ===== 367
<Response [200]>
===== ROW HASIL ===== 368
<Response [200]>
===== ROW HASIL ===== 369
<Response [200]>
===== ROW HASIL ===== 370
<Response [200]>
===== ROW HASIL ===== 371
<Response [200]>
===== ROW HASIL ===== 372
<Response [200]>
===== ROW HASIL ===== 373
<Response [200]>
===== ROW HASIL ===== 374
<Response [200]>
===== ROW HASIL ===== 375
<Response [200]>
===== ROW HASIL ===== 376
<Response [200]>
===== ROW HASIL ===== 377
<Response [200]>
===== ROW HASIL ===== 378
<Response [200]>
===== ROW HASIL ===== 379
<Response [200]>
===== ROW HASIL ===== 380
<_io.TextIOWrapper name='/content/drive/MyDrive/Semester_5/Information_Extraction/metaPidanaUmumPNwonogiri.csv' mode='w'
tidak ada
Time Processing : 1160.20470547676 Second

```

Fungsi main(), fungsi utama untuk membaca daftar URL putusan, scraping metadata tiap URL, lalu menyimpannya ke CSV.

```

fileListURL = "/content/drive/MyDrive/Semester_5/Information_Extraction/hasilListURLPage.txt"
fileMetaCSV = "/content/drive/MyDrive/Semester_5/Information_Extraction/metaPidanaUmumPNwonogiri.csv"

```

- fileListURL → file .txt berisi daftar URL putusan.
- fileMetaCSV → file .csv tempat menyimpan hasil metadata.

Prosesnya:

Buka file URLTest.txt dan baca semua baris (setiap baris = 1 URL).

Untuk setiap URL, panggil generateMeta(url) untuk mengambil metadata.

Simpan hasilnya ke dalam list listHasil.

Setelah selesai, panggil generateFileCSV(listHasil, fileMetaCSV) untuk menyimpan hasil ke CSV.

```

for barisURL in bacaListURL:
    try:
        hasil = generateMeta(str(barisURL))
        listHasil.append(hasil)
    except Exception as e:
        print(f"Error Get Meta Inf, {e}")

```

Kalau ada error (misalnya halaman tidak bisa dibuka), program tidak berhenti, tapi lanjut ke URL berikutnya.

Terakhir, ditampilkan lama waktu proses dengan time.time().

